EX03_CSP_GAME
Advanced Artificial Intelligence



GV HƯỚNG DẪN : Thầy Đỗ Như Tài
HỌ VÀ TÊN SV: Bùi Minh Trung
MSSV-LỚP: 3124720074 – DNT1241

BÀI TẬP – TÌM KIẾM, CSP VÀ GAME TRONG AI

. Phân tích đề bài

Mục	Nội dung
Mục tiêu	Minimax/Alpha-Beta; cắt tỉa trong cây có chance node; câu hỏi đúng/sai Minimax–Expectimax; Forward Checking, Arc Consistency trên đồ thị ràng buộc; trò chơi Pacman–Casper; tính đầy đủ/tối ưu của thuật toán tìm kiếm; A* khi chi phí cạnh đổi; tác tử và môi trường.
Yêu cầu trình bày	Lập luận ngắn gọn, nêu các bước tính. Câu có hình có thể điền vào hình hoặc làm bảng (mình làm cả hình và bảng). Ghi Họ tên/MSSV ở bảng trên.
Dữ liệu đọc từ hình	Bài 1: 18 lá = 3,0,−1,1,4,6 | 10,−1,0,−2,12,−3 | −4,−2,−2,10,−5,−7. Bài 2: vị trí ô viền đậm = lá 7 (cây 1), lá 8 (cây 2), lá 6 (cây 3). Bài 4: cạnh (A,B),(A,C),(A,D),(B,C),(B,E),(C,F); Hình 3 là đường G–E–C–A–B–D–F. Bài 5: các cặp (u,v) đã cho trong Hình 4. Hãy đối chiếu lại với bản gốc.
Quy ước nộp	Đề không nêu quy tắc đặt tên file và thang điểm; bạn đổi tên file theo quy định của lớp.

Bài 1. Minimax và Alpha-Beta Pruning

a) Giá trị các nút trung gian (Minimax)

Tầng	Nút (trái → phải)	Cách tính	Giá trị
MAX (tầng 2)	9 nút, mỗi nút 2 lá	max(3,0) | max(−1,1) | max(4,6) | max(10,−1) | max(0,−2) | max(12,−3) | max(−4,−2) | max(−2,10) | max(−5,−7)	3, 1, 6 | 10, 0, 12 | −2, 10, −5
MIN (tầng 1)	3 nút	min(3,1,6) | min(10,0,12) | min(−2,10,−5)	1 | 0 | −5
MAX (gốc)	1 nút	max(1, 0, −5)	**1**

Đáp án (a): các nút MAX = 3, 1, 6, 10, 0, 12, −2, 10, −5; các nút MIN = 1, 0, −5.

b) Nhánh MAX nên chọn và giá trị trò chơi

Đáp án (b): giá trị các nhánh là 1, 0, −5 nên MAX chọn **nhánh trái** (nút MIN thứ nhất); **giá trị trò chơi = 1**.

c) Alpha-Beta (trái → phải)

Quy tắc: nút MAX cắt khi v ≥ β; nút MIN cắt khi v ≤ α. α, β ở mỗi nút là giá trị lúc vào nút; diễn biến cập nhật ghi ở cột kế bên (hình vẽ ở trang cuối phần Bài 5, Hình 1).

Nút	α vào	β vào	Diễn biến	Trả về
Gốc (MAX)	−∞	+∞	α: −∞ → 1 sau nhánh trái; hai nhánh còn lại không vượt 1	1
MIN 1	−∞	+∞	β: +∞ → 3 → 1	1
MAX 1.1	−∞	+∞	lá 3 → v = 3, α = 3; lá 0 → v = 3	3
MAX 1.2	−∞	3	lá −1 → α = −1; lá 1 → v = 1, α = 1	1
MAX 1.3	−∞	1	lá 4 → v = 4 ≥ β = 1 ⇒ **cắt lá 6**	4
MIN 2	1	+∞	β: +∞ → 10 → 0; v = 0 ≤ α = 1 ⇒ **cắt MAX 2.3**	0
MAX 2.1	1	+∞	lá 10 → v = 10, α = 10; lá −1 → v = 10	10
Nút	**α vào**	**β vào**	**Diễn biến**	**Trả về**

MAX 2.2	1	10	lá 0 → v = 0; lá −2 → v = 0	0
MIN 3	1	+∞	β: +∞ → −2; v = −2 ≤ α = 1 ⇒ **cắt MAX 3.2, 3.3**	−2
MAX 3.1	1	+∞	lá −4 → v = −4; lá −2 → v = −2	−2

Đáp án (c): các nhánh đánh dấu X: lá 6 (dưới MAX 1.3); nút MAX 2.3 (lá 12, −3); nút MAX 3.2 (lá −2, 10) và MAX 3.3 (lá −5, −7). Giá trị gốc = 1.

d) Các lá không cần thăm và so sánh

Đáp án (d): lá không cần thăm: **6, 12, −3, −2, 10, −5, −7** (7 lá). Minimax đánh giá **18/18 lá**; Alpha-Beta chỉ đánh giá **11/18 lá** (3, 0, −1, 1, 4, 10, −1, 0, −2, −4, −2), tiết kiệm 7 lá (≈ 39%) mà giá trị và nước đi gốc không đổi.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Hướng giải. Chance node C có 2 con xác suất đều nhau: C = (x + y)/2. Một chance node chỉ cho phép cắt tỉa nếu biết được *cận* của giá trị nó cần có. Khi các lá chưa biết giới hạn, ta chỉ có thể cắt **con cuối cùng** của chance node, vì khi đó mọi con còn lại đã biết nên ngưỡng được tính chính xác: nếu C cần < β thì con cuối phải < β′ = 2β − (giá trị con trước). Con **đầu** của chance node thì con còn lại chưa biết (có thể nhỏ tùy ý) nên không có ngưỡng. Quy ước: không cắt khi bằng nhau, nên dùng bất đẳng thức chặt.

a) Trường hợp 1 (A MIN → B MAX → C chance); ô đậm = lá 7

Để cắt lá 7 phải cắt cả C thứ tư (lá 7, 8) bên dưới B2. Sau khi xét B1, A (nút MIN) có β = B1. Tại B2 (nút MAX), xét C3 xong nếu C3 ≥ β thì B2 ≥ β, A sẽ không chọn B2 nên cắt C4. Điều kiện: **(l + l)/2 > B1 = max((l +l)/2, (l +l)/2)**.

Đáp án (a): khả thi. Ví dụ l = 2, 4, 1, 3, 5, 7: C1 = 3, C2 = 2 ⇒ B1 = 3 = β(A); C3 = 6 > 3 ⇒ B2 ≥ 6 > 3 ⇒ cắt C4 (lá 7, 8), A = 3.

b) Trường hợp 2 (A MIN → C chance → B MAX); ô đậm = lá 8

A có β = C1 sau khi xét C1. Với C2 = (B3 + B4)/2, C2 chỉ có ích nếu C2 < C1, tức B4 < β′ = 2·C1 − B3. B4 là con *cuối* của C2 nên β′ xác định. Tại B4 (MAX), nếu lá 7 ≥ β′ thì B4 ≥ β′ và C2 ≥ C1 nên cắt lá 8. Điều kiện: **l > 2·C1 − B3**, tức (B3 + l)/2 > C1.

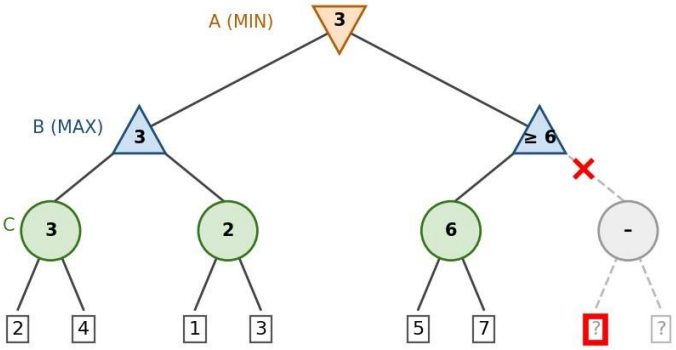
Đáp án (b): khả thi. Ví dụ l = 2, 4, 3, 1, 1, 2, 6: B1 = 4, B2 = 3 ⇒ C1 = 3.5; B3 = 2 ⇒ β′ = 2·3.5 − 2 = 5; l = 6 > 5 ⇒ C2 ≥ (2+6)/2 = 4 > 3.5 ⇒ cắt lá 8.

c) Trường hợp 3 (cùng cấu trúc trường hợp 2); ô đậm = lá 6

Lá 6 là lá thứ hai của B3, mà B3 là con *đầu* của chance node C2. Để cắt lá 6 cần chứng minh B3 vượt một ngưỡng β′, mà β′ = 2β − B4 với B4 chưa biết (có thể nhỏ tùy ý, nên β′ có thể lớn tùy ý). Do đó không có giá trị hữu hạn nào của lá 1–5 làm lá 6 chắc chắn vô ích: dù B3 lớn đến đâu, B4 rất nhỏ vẫn có thể kéo C2 xuống dưới C1.

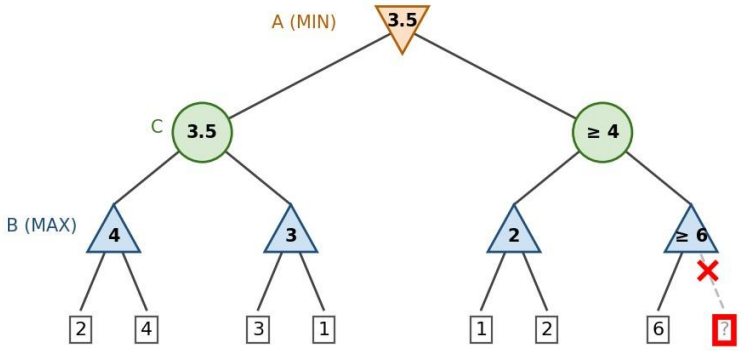
Đáp án (c): Không thể (không có cận cho giá trị con chưa xét, nên không thể cắt lá 6 khi chưa có giới hạn giá trị lá).

Trường hợp 1: khả thi (lá 7 viền đậm bị cắt cùng cả C thứ tư)



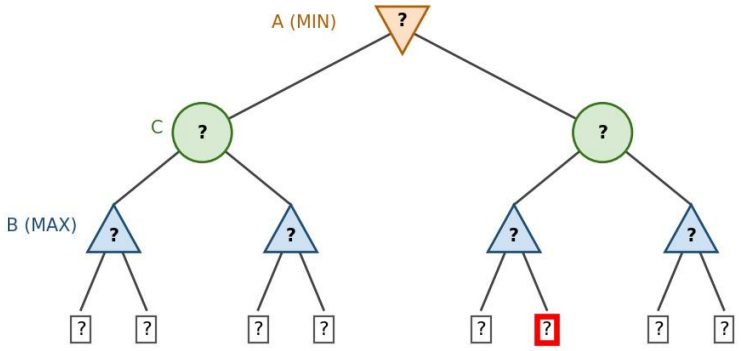
B1 = 3 ⇒ β(A) = 3
C3 = 6 ≥ β = 3 ⇒ B2 ≥ 6 > 3
⇒ cắt C4 (lá 7, 8)

Trường hợp 2: khả thi (lá 8 viền đậm bị cắt)



β(A) = C1 = 3.5; B3 = 2 ⇒ ngưỡng cho B4: β′ = 2·3.5 − 2 = 5
l7 = 6 > 5 ⇒ B4 ≥ 6 ⇒ C2 ≥ 4 > 3.5 ⇒ cắt l8

Trường hợp 3: Không thể (lá 6 viền đậm)



KHÔNG THỂ: ngưỡng của B3 là β′ = 2β − B4 với B4 chưa biết
(có thể nhỏ tùy ý) ⇒ không có cận ⇒ không cắt được l6

Hình 2. Ba trường hợp: ví dụ gán giá trị lá (trường hợp 1, 2) và lý do không thể (trường hợp 3). ✕ = nhánh bị cắt, ô viền đỏ = ô viền đậm của đề.

d) Vì sao khác Alpha-Beta trong cây Minimax?

(1) Min/max chỉ phụ thuộc *thứ tự* so sánh: một con đã thấy là đủ để chặn giá trị của nút (MAX ≥ con đã xét), nên cắt được ngay. (2) Chance node là *trung bình có trọng số*, phụ thuộc giá trị cụ thể của mọi con; một con chưa xét có thể kéo trung bình lên/xuống tùy ý nếu không có cận. (3) Vì vậy chỉ cắt được khi đã biết hết các con còn lại (con cuối) hoặc khi có cận giá trị lá (cận trên + xác suất còn lại), và cửa sổ (α, β) truyền xuống phải biến đổi: β′ = (β − Σ pv)/p . (4) Do đó cắt tỉa trong cây có chance node kém hiệu quả

i i k

hơn nhiều so với cây Minimax.

> **Đáp án (d):** min/max cho cận chỉ từ một con, trung bình thì không; cắt tỉa chỉ khả thi với con cuối của chance node hoặc khi biết giới hạn giá trị lá.

Bài 3. Câu hỏi đúng/sai

a) Luôn có V ≤ V

M E

☒ Đúng ☐ Sai Chứng minh quy nạp theo độ cao: lá có V = V . Nút MAX: max đơn điệu nên nếu mọi con có V ≥ V thì

E M E M

cả nút cũng vậy. Nút MIN thay bằng chance: V = Σ pV (c) ≥ Σ pV (c) ≥ min V (c) = V (kỳ vọng không

E i E i i M i i M i M

nhỏ hơn giá trị nhỏ nhất).

b) Chính sách Minimax tối ưu đảm bảo lợi ích kỳ vọng ≥ V

M

☒ Đúng ☐ Sai Chính sách Minimax bảo đảm kết quả ≥ V trước *mọi* hành vi của đối thủ (kể cả ngẫu nhiên). Khi đối thủ

M

chỉ chọn theo phân phối xác suất, mỗi lần chơi cho tiện ích ≥ V nên kỳ vọng cũng ≥ V .

M M

c) Chính sách Minimax tối ưu đảm bảo lợi ích kỳ vọng ≥ V

E

☐ Đúng ☒ Sai

Chính sách Minimax là bi quan, không tối ưu cho cây có chance node. **Phản ví dụ:** gốc MAX có hai hành động: a → nút MIN với kết quả {1, 1}; b → nút MIN với kết quả {0, 100}. V = max(min(1,1), min(0,100)) =

M

1 ⇒ chọn a. Thay MIN bằng chance 50/50: a cho 1, b cho 50 ⇒ V = 50. Chính sách Minimax (a) chỉ được kỳ

E

vọng 1 < 50 = V . Tổng quát: V ≤ (kỳ vọng của chính sách Minimax) ≤ V .

E M E

Đáp án: a) Đúng; b) Đúng; c) Sai.

Trò chơi một người có kết quả hành động không xác định (+1 thắng, −1 thua)

Phương pháp	Chọn?	Giả định cần thiết
Minimax với tiện ích +1/−1	☒	Coi mọi kết quả ngẫu nhiên là do "đối thủ tưởng tượng" chọn bất lợi nhất (worst case). Chỉ cần biết các kết quả có thể xảy ra, không cần xác suất; cho giá trị bảo đảm bi quan (thắng chắc hay không).
Expectimax với tiện ích +1/−1	☒	Biết xác suất của các kết quả; cây hữu hạn/không chu trình. Giá trị = P(thắng) − P(thua), tối đa hóa kỳ vọng.
Value Iteration (phần thưởng 0 trung gian, ±1 ở kết quả)	☒	Mô hình MDP: biết hàm chuyển T(s,a,s') và phần thưởng; dùng được cả khi có chu trình/vô hạn bước (γ = 1 nếu trò chơi kết thúc với xác suất 1, hoặc γ < 1). Cho cùng giá trị với Expectimax khi cây hữu hạn.
Không phương án nào	☐	Sai, vì ba cách trên đều dùng được với giả định tương ứng.

d) Giải thích

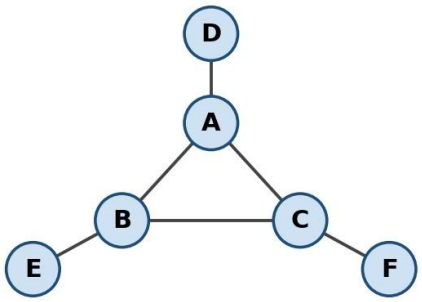
Đáp án (d): cả ba phương pháp đều phù hợp, tùy giả định: Minimax khi muốn bảo đảm trường hợp xấu nhất (không cần xác suất); Expectimax và Value Iteration khi biết xác suất kết quả và muốn tối đa hóa kỳ vọng; Value Iteration tổng quát hơn vì xử lý được trạng thái lặp.

Bài 4. CSP với Constraint Graph và lan truyền ràng buộc

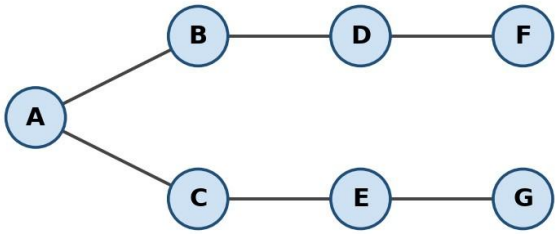
a) Đồ thị ràng buộc

Đỉnh A..F; cạnh: A–B, A–C, A–D, B–C, B–E, C–F. Tam giác A–B–C và ba đỉnh "treo" D (vào A), E (vào B), F (vào C).

4a. Đồ thị ràng buộc



Hình 3: A–B–D–F và A–C–E–G (thực chất là một đường đi G–E–C–A–B–D–F)



Hình 4a. Đồ thị ràng buộc câu (a) và đồ thị Hình 3 dùng cho câu (f)–(h).

Câu	Phân tích	Đáp án
b	Forward Checking tại A chỉ lọc miền của biến *chưa gán kề với A*.	**B, C, D**
c	Sau A đã gán, gán B: FC lọc các biến chưa gán kề B: C và E (A đã gán, D không kề B).	**C, E**
d	Duy trì AC (lan truyền): trực tiếp B, C, D; sau đó B, C co lại thì kéo theo các cung (E,B), (F,C), (B,C), (C,B); D không còn hàng xóm khác nên dừng.	**B, C, D, E, F** (E, F chỉ đổi nếu B, C bị co)
e	Sau B đã gán: AC lan từ B tới C và E, rồi từ C tới F. D chỉ kề A (đã gán và đã nhất quán) nên không đổi.	**C, E, F** (D không đổi)

Thuật toán mới: AC ban đầu và sau mỗi lần gán xong 2, 4, 6, … biến

Giả định. (1) Đồ thị Hình 3 là cây, thực chất là đường đi **G–E–C–A–B–D–F**. (2) Bài toán có nghiệm; AC ban đầu thành công. (3) Quay lui kiểm tra ràng buộc với các biến đã gán; AC (tính cả biến đã gán như miền một giá trị) chạy sau lần gán thứ 2, 4, 6. (4) 1 backtrack = 1 lần hoàn tác một phép gán do thất bại (AC phát hiện miền rỗng hoặc hết giá trị). (5) Miền cỡ d, trường hợp xấu nhất = thử sai tất cả trước khi đúng.

Hai tính chất của CSP dạng cây khi AC thành công: (L1) mọi giá trị còn lại trong mọi miền đều thuộc một nghiệm. (L2) nếu tập biến đã gán là *liên thông* và các giá trị tương thích với nhau, thì luôn mở rộng được, vì giá trị đã gán có điểm hỗ trợ (support) ở biến kề. Hệ quả: nếu hai lần gán liên tiếp giữa hai lần chạy AC cùng nằm trong một tập liên thông thì không bao giờ thất bại. Thất bại chỉ xảy ra khi hai biến được gán giữa hai lần AC là *không kề nhau, tách rời* và kết hợp giá trị của chúng không mở rộng được (AC ngay sau đó mới phát hiện).

Thứ tự	Tập đã gán sau mỗi bước (liên thông?)	Phân tích	Số backtrack xấu nhất
f) A→B→C→D→E→F→G	{A}, {A,B}, {A,B,C}, {A,B,C,D}, … luôn liên thông (mỗi biến kề một biến đã gán).	Mỗi biến mới có giá trị tương thích với hàng xóm đã gán nhờ AC; AC sau bước 2, 4, 6 không bao giờ làm rỗng miền.	**0**
g) G→E→C→A→B→D→F	Đi dọc đường G–E–C–A–B–D–F: luôn liên thông.	Giống (f).	**0**
h) G→F→E→D→C→B→A	{G,F} tách rời (hai đầu đường đi); {G,F,E,D}; {…,C,B} còn hở ở A.	Ba cặp (G,F), (E,D), (C,B), mỗi cặp là hai biến ở hai đầu hở nên có thể xung đột sau AC. Sau mỗi AC thành công, với giá trị bất kỳ của biến lẻ (thuộc một nghiệm theo L1) luôn tồn tại ít nhất 1 trong d giá trị của biến chẵn tương thích ⇒ tối đa d − 1 lần thử sai, mỗi lần AC làm rỗng miền và phải hoàn tác biến chẵn. Biến cuối A có miền đã nhất quán với hai bên ⇒ không thất bại.	**≤ 3(d − 1)** (cận trên); mô phỏng: 1, 3, 5 với d = 2, 3, 4

Đáp án (f, g, h): f) **0** backtrack; g) **0** backtrack; h) **không quá 3(d − 1)** backtrack (cận trên từ lập luận ở trên, ví dụ d = 3 ⇒ 6). Khi mình mô phỏng thuật toán trên đường G–E–C–A–B–D–F với các quan hệ ràng buộc ngẫu nhiên và tìm kiếm định hướng, số backtrack lớn nhất tìm được là 1, 3, 5 với d = 2, 3, 4 (tức nhỏ hơn cận trên, khoảng 2d − 3); cận 3(d − 1) có thể chưa chặt vì sau mỗi lần AC miền co lại. Nhận xét: AC chỉ sau mỗi hai lần gán chỉ gây quay lui khi hai biến gán giữa hai lần AC không nằm cùng tập liên thông; nếu AC sau *mọi* lần gán thì cây không cần quay lui với mọi thứ tự.

Bài 5. Cây trò chơi Pacman và Casper

Quy ước đọc hình. Hình ngũ giác = nút Pacman (MAX), hình thang = nút Casper. Các cặp đã cho ở nút Casper như (3,9), (4,8), (4,6)… cho thấy quy tắc: **Casper: u = min các u con (Pacman tin Casper đối kháng); v = max các v con (Casper thật sự muốn tăng tiện ích)**. **Pacman: u = max các u con; v = v của con có u lớn nhất** (Pacman hành động theo niềm tin u, nên giá trị thực là v của nhánh được chọn). **a) Điền các cặp (u, v)**

Nút	Con	u	v	Cặp
Pacman (cháu của (2,6)): con (4,6), (1,7)	(4,6), (1,7)	max(4,1) = 4	v của (4,6) = 6	**(4, 6)**
Casper trái: con (2,2), (4,6)	(2,2), (4,6)	min(2,4) = 2	max(2,6) = 6	**(2, 6)**
Pacman: con (0,6), (3,5)	(0,6), (3,5)	max(0,3) = 3	v của (3,5) = 5	**(3, 5)**
Casper giữa: con (1,1), (3,5)	(1,1), (3,5)	min(1,3) = 1	max(1,5) = 5	**(1, 5)**
Pacman tầng 1 trái: con (2,6), (1,5)	(2,6), (1,5)	max(2,1) = 2	v của (2,6) = 6	**(2, 6)**
Pacman tầng 1 phải: con (3,9), (4,8)	(3,9), (4,8)	max(3,4) = 4	v của (4,8) = 8	**(4, 8)**
Gốc (Pacman): con (2,6), (4,8)	(2,6), (4,8)	max(2,4) = 4	v của (4,8) = 8	**(4, 8)**

Đáp án (a): ngũ giác (cháu trái) = (4,6); thang trái = (2,6); ngũ giác giữa = (3,5); thang giữa = (1,5); hai nút tầng 1 = (2,6) và (4,8); **gốc = (4,8)**. **Khác biệt u và v tại nút Casper:** u là giá trị Pacman *tin* (Casper chọn con làm tiện ích nhỏ nhất ⇒ min), còn v là giá trị *thực* (Casper thân thiện chọn con làm tiện ích lớn nhất ⇒ max). Vì thế u ≤ v tại mọi nút Casper, và Pacman đi theo u nhưng nhận kết quả v.

Ghi chú: nếu coi nút gốc là Casper (thay vì Pacman) thì gốc sẽ là (min(2,4), max(6,8)) = (2, 8); các nút còn lại không đổi.

b) Các nhánh có thể không cần duyệt (trái → phải)

Pacman quyết định theo u nên tính u bằng Alpha-Beta (Pacman = MAX, Casper = MIN); v chỉ cần dọc theo nhánh được chọn. Quá trình: Casper trái: lá (2,2) cho β = 2. Xét nút Pacman con: (4,6) có u = 4 ≥ β = 2 ⇒ **cắt (1,7)** (cùng 2 lá). Sau đó nhánh Casper giữa có α = 2 và lá (1,1) cho u = 1 ≤ α ⇒ **cắt toàn bộ nút Pacman (3,5)** cùng các con (0,6), (3,5) và 4 lá bên dưới. Nhánh phải đều cần duyệt (α = 2 không đủ để cắt: u = 3 và 4 > 2).

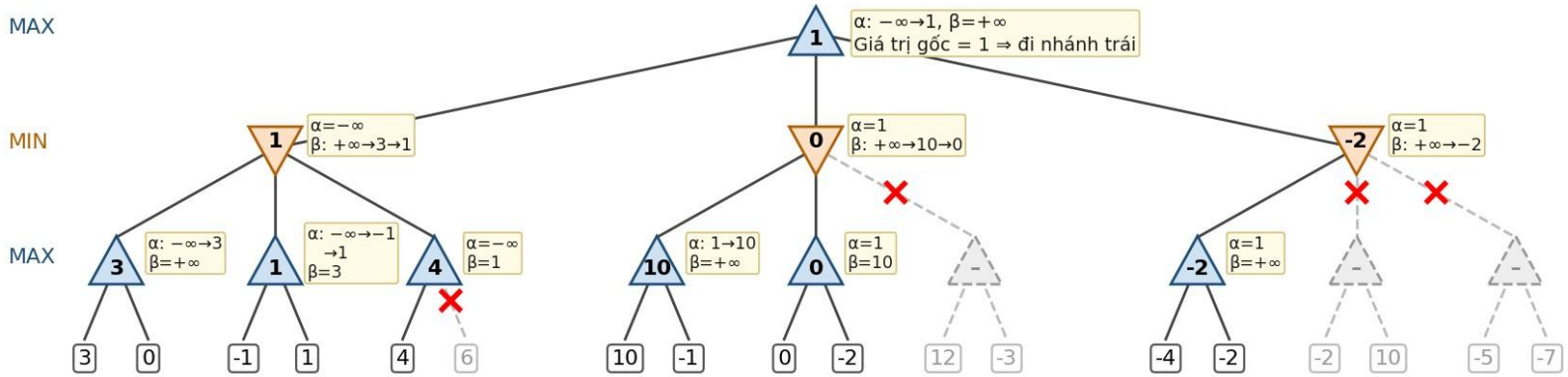
Đáp án (b): bỏ qua (đánh dấu X): nút Casper **(1,7)** với hai lá (1,1), (7,7); nút Pacman **(3,5)** với các nút (0,6), (3,5) và các lá (6,6), (0,0), (3,3), (5,5). Tổng cộng 6/14 lá không cần thăm; giá trị gốc vẫn (4,8).
Hình 5 ở trang kế tiếp.

c) Nếu Pacman biết Casper thân thiện

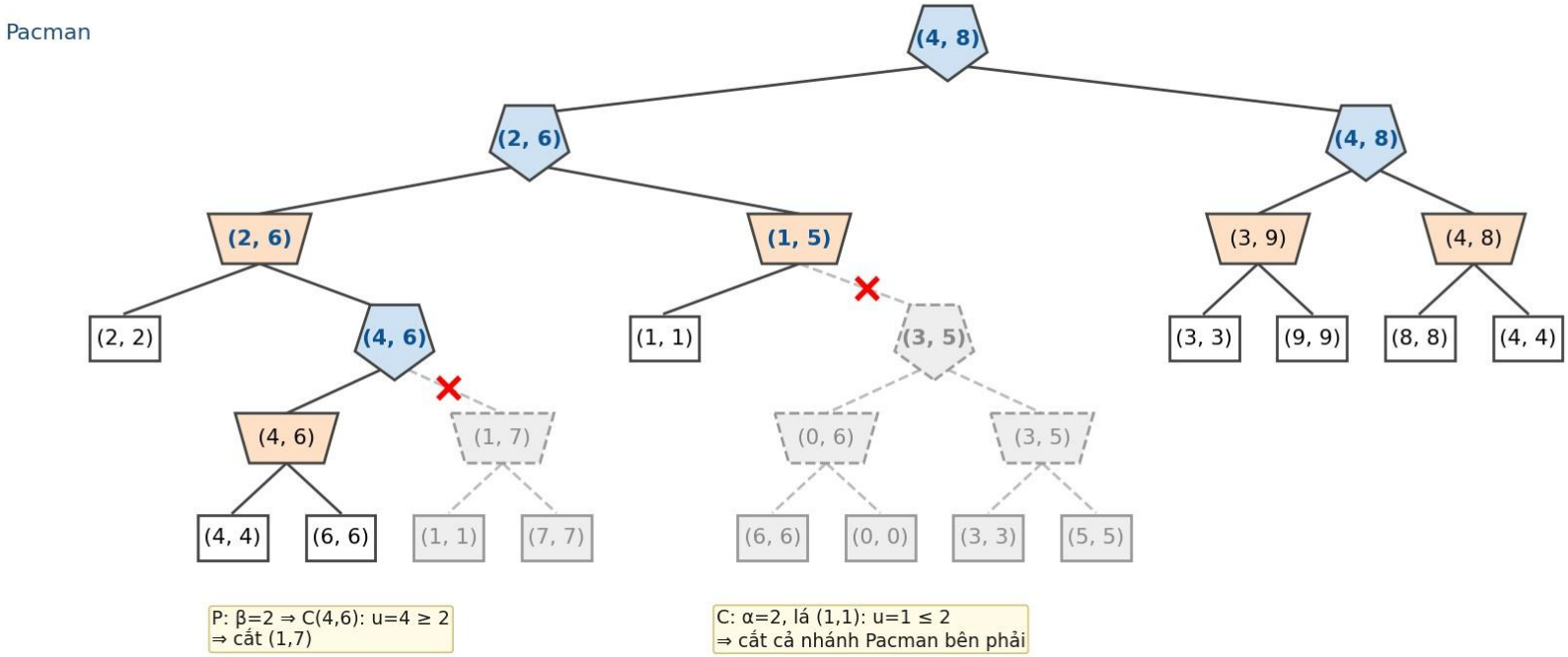
Khi đó Pacman dùng u = v: mọi nút Casper cũng là MAX. Các giá trị: (4,6)→6 ở nút Pacman cháu; Casper trái = max(2, max(6,7)) = 7; nhánh giữa = max(1, max(6,5)) = 6; tầng 1 trái = 7; Casper (3,9) = 9, (4,8) = 8 ⇒ tầng 1 phải = 9. Gốc = max(7, 9) = 9 (đi nhánh phải, rồi vào nút (3,9) để tới lá 9).

Đáp án (c): giá trị toàn cây tăng từ **8** (khi Pacman hiểu nhầm) lên **9** (bằng lá lớn nhất 9): Pacman sẽ chọn nút Casper (3,9) thay vì (4,8) vì biết ma sẽ giúp lấy lá 9. Hiểu nhầm làm Pacman mất 1 đơn vị tiện ích.

Hình lời giải Bài 1 (c) và Bài 5



Hình 1. Alpha-Beta Pruning: α, β ở mỗi nút (hộp vàng), giá trị trả về ghi trong nút, ✕ = nhánh bị cắt, nút/lá xám = không được thăm.



Hình 5. Cây Pacman–Casper đã điền (số xanh đậm = cặp (u,v) mình tính; ✕ = nhánh có thể bỏ qua khi tính u bằng Alpha-Beta).

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Thuật toán	Điều kiện đầy đủ (Complete)	Điều kiện tối ưu (Optimal)
BFS	Hệ số nhánh b hữu hạn (và có đích ở độ sâu hữu hạn). Đúng cho cả tìm kiếm trên cây và trên đồ thị.	Mọi chi phí bước bằng nhau (hoặc chi phí đường đi không giảm theo độ sâu); khi đó nút đích nông nhất cũng rẻ nhất.
DFS	**Cây:** chỉ đầy đủ khi không gian cây hữu hạn (độ sâu hữu hạn, không chu trình/nhánh vô hạn). **Đồ thị:** đầy đủ khi không gian trạng thái hữu hạn (có tập đã thăm).	Không tối ưu nói chung (trả về đích đầu tiên tìm được, có thể sâu và đắt hơn).
IDS	b hữu hạn (như BFS); không bị kẹt trong nhánh vô hạn nhờ giới hạn độ sâu.	Chi phí bước bằng nhau (như BFS); với chi phí khác nhau không bảo đảm.
A*	b hữu hạn và mọi chi phí bước $\geq \varepsilon > 0$ (hoặc tổng chi phí hữu hạn); có đường tới đích.	**Cây:** heuristic chấp nhận được (admissible: $0 \leq h \leq h^*$). **Đồ thị:** heuristic nhất quán (consistent: $h(n) \leq c(n,n') + h(n')$), hoặc admissible nhưng cho phép mở lại nút đã đóng.

Vì sao chi phí bước và heuristic ảnh hưởng? BFS/IDS mở rộng theo số bước, không theo chi phí, nên chỉ tối ưu khi mọi bước cùng giá; khi chi phí khác nhau cần UCS hoặc A*. Chi phí bước 0 hoặc tiến về 0 tạo vô hạn đường đi rẻ, làm A*/UCS có thể không đầy đủ, nên cần $\varepsilon > 0$; chi phí âm phá vỡ lập luận "nút lấy ra đầu tiên là tốt nhất". Với A*, heuristic đánh giá thấp (admissible) bảo đảm khi đích được lấy ra khỏi hàng đợi thì không còn đường rẻ hơn ($f = g + h \leq$ chi phí thật). Trên *đồ thị* nút có thể được đóng sớm theo một đường đắt; heuristic chỉ admissible có thể khiến tìm ra đường không tối ưu, còn consistent đảm bảo g của nút khi đóng đã tối ưu và f không giảm dọc đường. Heuristic quá lớn (không admissible) làm A* có thể bỏ qua đường tối ưu; heuristic càng sát h* thì càng mở rộng ít nút.

> **Đáp án:** BFS, IDS: đầy đủ khi b hữu hạn; tối ưu khi chi phí bước đều nhau. DFS: đầy đủ chỉ khi không gian hữu hạn/có tập đã thăm; không tối ưu. A*: đầy đủ khi b hữu hạn và chi phí $\geq \varepsilon > 0$; tối ưu với heuristic admissible (cây) hoặc consistent (đồ thị).

Bài 7. A* khi chi phí một cạnh thay đổi

Ký hiệu: C* = chi phí tối ưu cũ, P* = đường tối ưu cũ. Mọi đường qua cạnh X→Y đổi chi phí đúng ±n (mỗi đường dùng cạnh một lần); đường không dùng cạnh giữ nguyên chi phí. A* đã mở rộng mọi nút có $f < C^*$ (heuristic admissible/consistent) và các nút còn lại trong hàng đợi có $f \geq C^*$.

Câu	Chi phí tối ưu mới	Đường tối ưu	Cách cập nhật (không nhất thiết chạy lại từ đầu)
a) Tăng n; cạnh thuộc P*, đã khám phá	Tăng, nằm trong $[C^*, C^* + n]$: bằng $\min(C^* + n$, chi phí đường thay thế tốt nhất).	Có thể đổi: nếu có đường khác có chi phí $< C^* + n$ thì đường đó thay thế P*.	Cộng n vào g của Y và mọi nút hậu duệ của Y trong cây tìm kiếm (các nút có đường qua X→Y), cập nhật f, đưa lại vào hàng đợi; tiếp tục A* với hàng đợi và tập đã khám phá hiện có cho tới khi lấy được đích; chỉ cần mở thêm các nút có $f < C^* + n$. Heuristic vẫn admissible (chi phí thật chỉ tăng).
b) Giảm n; cạnh thuộc P*, đã khám phá	Giảm đúng n: **$C^* - n$**.	Không đổi: P* giảm n, đường khác giảm tối đa n (nếu dùng cạnh đó, vẫn $\geq C^* - n$) hoặc không đổi ($> C^* - n$).	Chỉ cập nhật chi phí = $C^* - n$, không cần tìm lại. (Heuristic có thể thành không admissible, nhưng đường P* vẫn tối ưu nên không ảnh hưởng.)
c) Tăng n; cạnh không thuộc P*, đã khám phá	Không đổi (C*).	Không đổi.	Không cần làm gì: P* không đi qua cạnh, mọi đường khác chỉ đắt thêm hoặc bằng.

Câu	Chi phí tối ưu mới	Đường tối ưu	Cách cập nhật (không nhất thiết chạy lại từ đầu)
d) Giảm n; cạnh không thuộc P*, đã khám phá	Giảm 0 đến n: $C^*_{mới} = \min(C^*, g(X) + (c - n) + h^*(Y))$, tức nằm trong $[C^* - n, C^*]$.	Có thể đổi sang đường mới đi qua X→Y.	Gán lại $g(Y) = g(X) + c - n$ nếu nhỏ hơn g cũ; lan truyền (relax) giảm g tới các hậu duệ của Y trong phần đã khám phá, mở lại các nút đã đóng có g giảm và đưa vào hàng đợi; tiếp tục A* tới khi đích lấy ra có f ≥ chi phí tốt nhất hiện có. Giả sử heuristic vẫn hợp lệ.
e) Tăng n; cạnh không thuộc P*, chưa khám phá	Không đổi.	Không đổi.	Không cần làm gì: cạnh chưa bao giờ được dùng và chỉ đắt thêm.
f) Giảm n; cạnh không thuộc P*, chưa khám phá	Không đổi (nếu heuristic vẫn admissible).	Không đổi.	X chưa được mở rộng nên $f(X) = g(X) + h(X) \geq C^*$. Với h vẫn admissible sau thay đổi, mọi đường qua X→Y có chi phí $\geq g(X) + h(X) \geq C^*$, nên không cải thiện. Chỉ khi heuristic trở nên không hợp lệ mới phải kiểm tra lại (đưa X vào mở rộng tiếp).

Tóm tắt: chi phí tối ưu có thể đổi ở (a) và (d), luôn đổi đúng −n ở (b), không đổi ở (c), (e), (f). Chỉ (a) và (d) cần cập nhật cục bộ (tăng/giảm g của hậu duệ rồi chạy tiếp A* trên hàng đợi cũ, theo tinh thần incremental như LPA*/D*), còn lại chỉ cần cập nhật chi phí hoặc giữ nguyên.

Bài 8. Tác tử thông minh và môi trường

Bài	Đáp án chính

1	MAX: 3,1,6|10,0,12|−2,10,−5; MIN: 1,0,−5; gốc = 1 (nhánh trái); cắt lá 6, MAX 2.3, MAX 3.2, MAX 3.3; thăm 11/18 lá

A = hành động không phụ thuộc percept (có thể ngẫu nhiên); B = phản xạ đơn giản (chỉ phụ thuộc percept hiện tại); C = phụ thuộc toàn bộ lịch sử percept (C bao hàm B, B bao hàm A về năng lực). Tác tử hợp lý hoàn hảo phải chọn hành động tối ưu theo thông tin mà nó có thể biết về trạng thái thực.

Môi trường	A	B	C	Giải thích
a) Quan sát đầy đủ, xác định, rời rạc, 1 tác tử, tĩnh	☐	☒	☒	Percept = trạng thái đầy đủ nên hành động tối ưu là một hàm của percept hiện tại (chính sách tối ưu dừng theo trạng thái), B đủ; C cũng làm được (bỏ qua lịch sử). A không đủ vì hành động tối ưu phụ thuộc trạng thái.
b) Quan sát một phần, xác định, rời rạc, 1 tác tử, tĩnh	☐	☐	☒	Percept hiện tại có thể giống nhau ở các trạng thái khác nhau cần hành động khác nhau. Trong môi trường xác định, lịch sử percept cho phép suy ra trạng thái (belief), nên chỉ C hợp lý hoàn hảo.
c) Quan sát một phần, ngẫu nhiên, rời rạc, 1 tác tử, tĩnh	☐	☐	☒	Hành động tối ưu phụ thuộc belief state, là hàm của lịch sử percept, nên C cần thiết; B, A không đủ. C có thể dùng hàm f xác định (có chính sách tối ưu xác định theo belief), nên không cần tự ngẫu nhiên hóa.

Đáp án: a) **B và C**; b) **chỉ C**; c) **chỉ C**.

d) Có tồn tại môi trường mà mọi tác tử đều hành động hợp lý?

☒ Đúng ☐ Sai

Đáp án (d): Đúng. Tính hợp lý nghĩa là tối đa hóa kỳ vọng độ đo hiệu quả. Nếu mọi hành động đều cho cùng độ đo hiệu quả thì mọi tác tử đều tối đa hóa nó. Ví dụ: môi trường chỉ có một hành động hợp lệ (no-op), hoặc môi trường mà phần thưởng luôn bằng 0 bất kể hành động; khi đó tác tử A, B, C, kể cả tác tử chọn ngẫu nhiên, đều hợp lý.

Tổng hợp đáp án

Bài	Đáp án chính
2	1) khả thi: C3 > B1 (ví dụ 2,4,1,3,5,7); 2) khả thi: l > 2·C1 − B3 (ví dụ 2,4,3,1,1,2,6); 3) Không thể; chance 7 node chỉ cắt được con cuối/khi có cận
3	a) Đúng; b) Đúng; c) Sai; trò chơi một người: Minimax, Expectimax, Value Iteration đều phù hợp với giả định tương ứng
4	b) B, C, D; c) C, E; d) B, C, D, E, F; e) C, E, F; f) 0; g) 0; h) ≤ 3(d − 1) (cận trên; mô phỏng thấy 1, 3, 5 với d = 2, 3, 4)
5	(2,6) và (4,6),(3,5),(1,5) ở các nút giữa, hai nút tầng 1 = (2,6),(4,8), gốc (4,8); cắt (1,7) và nhánh (3,5); nếu Pacman biết ma thân thiện thì giá trị 8 → 9
6	BFS/IDS: b hữu hạn, chi phí đều; DFS: không gian hữu hạn, không tối ưu; A*: ε > 0, admissible (cây)/consistent (đồ thị)
7	a) tăng ≤ n, cập nhật g hậu duệ; b) C* − n; c) không đổi; d) giảm 0..n, relax; e) không đổi; f) không đổi
8	a) B, C; b) C; c) C; d) Đúng (ví dụ phần thưởng luôn bằng nhau)